

the same operator used with strings, it calls the `__add__()` method in the string class, which performs string concatenation.

This mechanism is generalized so that when Python encounters a `+` operator when working with objects instantiated from your class, it will call the `__add__()` method if one is present in your class. Therefore, as the class developer, you can write code to invent a new meaning for this operator.

Each operator maps to a specific magic method name. While there are many types of magic methods, let's start with those related to the comparison operators.

Comparison Operator Magic Methods

Consider our `Square` class from [Listing 9-2](#). You want client software to be able to compare two `Square` objects to see if they are equal. It is up to you to decide what “equal” means when comparing objects. For instance, you might define it as two objects being the same color and at the same location and with the same size. As a simple example, we'll define two `Square` objects as equal if they just have the same side length. This is easily implemented by comparing the `self.heightAndWidth` instance variables of the two objects and returning a Boolean. You could write your own `equals()` method, and client software could then call it like this:

```
if oSquare1.equals(oSquare2):
```

This would work fine. However, it would be more natural for client software to use the standard `==` comparison operator:

```
if oSquare1 == oSquare2:
```

Written this way, Python translates the `==` operator into a call to a magic method of the first object. In this case, Python will attempt to call a magic method named `__eq__()` in the `Square` class. [Table 9-1](#) shows the magic methods for all the comparison operators.